

Contexto

Don Donnie, dueño del **Casino Black Cat**, observando el auge de los casinos en línea y la transformación digital de la industria del entretenimiento, ha decidido modernizar su imperio para no quedarse atrás frente a la competencia, por lo que desea trasladar los juegos más emblemáticos de su casino al mundo digital, comenzando con la clásica **Ruleta**.

Como parte del equipo de desarrollo asignado al proyecto, tu misión es diseñar un *prototipo sencillo* en Java que funcione desde la línea de comandos. Este prototipo debe permitir a los jugadores realizar apuestas básicas (par/impar o rojo/negro), simular el giro de la ruleta y registrar los resultados.



Figura 1: Don Donnie - Dueño Casino

Objetivos

El programa a desarrollar debe permitir a los jugadores realizar apuestas básicas (par/impar o rojo/negro), simular el giro de la ruleta y registrar los resultados.

1. Crear un proyecto Java en **IntelliJ IDEA**.
2. Vincular el proyecto a un repositorio en **GitHub**.
3. Decidir el enfoque de trabajo: **Top-down** o **Bottom-up**.
4. Implementar un **menú de consola** que permita:
 - **Iniciar una ronda de ruleta:** el jugador elige el tipo de apuesta (par/impar o rojo/negro) y el monto. Luego, el programa simula el giro (número aleatorio entre 0 y 36), evalúa el resultado, registra los resultados en los arreglos **historialNumeros**, **historialApuestas**, **historialAciertos** y muestra si ganó o perdió.
 - **Ver estadísticas:** mostrar un resumen calculado a partir de los arreglos, por ejemplo:
 - Cantidad de rondas jugadas.
 - Total apostado
 - Total de Aciertos
 - % de acierto

- Ganancia/Pérdida neta
- Salir

5. Realizar **commits pequeños y con sentido**, evitando un único commit masivo.

Criterios de Evaluación

- Cada función debe encargarse de una única cosa (evitar el «método dios»).
- Cada función debe tener entre 5 a 10 líneas.
- Commits claros, uno por funcionalidad.
- El método main solo llama al menú (no contiene lógica).
- Uso de **final** para constantes (evitar números mágicos).
- Usar **arreglos unidimensionales** como estructura de datos principal.
- Los nombres de las funciones deben tener sentido y ser claros.

Código Base

```
import java.util.Random;
import java.util.Scanner;

public class Ruleta {

    public static final int MAX_HISTORIAL = 100;

    public static int[] historialNumeros = new int[MAX_HISTORIAL];
    public static int[] historialApuestas = new int[MAX_HISTORIAL];
    public static boolean[] historialAciertos = new boolean[MAX_HISTORIAL];
    public static int historialSize = 0;

    public static Random rng = new Random();
    public static int[] numerosRojos =
    {1,3,5,7,9,12,14,16,18,19,21,23,25,27,30,32,34,36};

    /**
     * Método principal: inicia el programa llamando al menú.
     */
}
```

```

public static void main(String[] args) {
    menu();
}

/**
 * Controla el flujo principal del programa mostrando un menú en consola.
 */
public static void menu() {}

/**
 * Muestra en consola las opciones disponibles del menú.
 */
public static void mostrarMenu() {}

/**
 * Lee la opción elegida por el usuario desde teclado.
 * @param in Scanner para entrada por consola.
 * @return número de opción ingresado.
 */
public static int leerOpcion(Scanner in) {
    return 0;
}

/**
 * Ejecuta la acción correspondiente a la opción del menú.
 * @param opcion opción elegida por el usuario.
 * @param in Scanner para entrada por consola.
 */
public static void ejecutarOpcion(int opcion, Scanner in) {}

/**
 * Inicia una ronda de la ruleta: leer apuesta, girar, evaluar y mostrar resultado.
 * @param in Scanner para entrada por consola.
 */
public static void iniciarRonda(Scanner in) {}

/**
 * Permite al usuario seleccionar el tipo de apuesta (R/N/P/I).
 * @param in Scanner para entrada por consola.
 */

```

```

    * @return el tipo de apuesta elegido.
    */
    public static char leerTipoApuesta(Scanner in) {
        return ' ';
    }

    /**
     * Simula el giro de la ruleta generando un número aleatorio de 0 a 36.
     * @return número de la ruleta.
     */
    public static int girarRuleta() {
        return 0;
    }

    /**
     * Evalúa si la apuesta realizada por el jugador fue acertada.
     * @param numero número obtenido en la ruleta.
     * @param tipo tipo de apuesta elegida.
     * @return true si acertó, false si perdió.
     */
    public static boolean evaluarResultado(int numero, char tipo) {
        return false;
    }

    /**
     * Determina si un número corresponde a color rojo.
     * @param n número de la ruleta.
     * @return true si es rojo, false en caso contrario.
     */
    public static boolean esRojo(int n) {
        return false;
    }

    /**
     * Registra los resultados de la ronda en los arreglos de historial.
     * @param numero número obtenido en la ruleta.
     * @param apuesta monto apostado.
     * @param acierto si el jugador acertó o no.
     */

```

```

public static void registrarResultado(int numero, int apuesta, boolean acierto) {}

/**
 * Muestra en consola el resultado de la ronda.
 * @param numero número obtenido en la ruleta.
 * @param tipo tipo de apuesta realizada.
 * @param monto monto apostado.
 * @param acierto si el jugador ganó o perdió.
 */
public static void mostrarResultado(int numero, char tipo, int monto, boolean
acierto) {}

/**
 * Muestra estadísticas generales de todas las rondas jugadas.
 */
public static void mostrarEstadisticas() {}
}

```

Listing 1: Esqueleto Java para Ruleta

Preguntas de reflexión

1. ¿Qué ventajas obtuviste al dividir la lógica en métodos cortos en lugar de un «método dios»?
2. ¿Cómo te ayudaron los arreglos a **listar y analizar resultados**?
3. ¿Qué diferencias notaste al trabajar con un enfoque Top-down frente a Bottom-up?

Referencias

- [1] Apuntes de Java. <https://jessustm.github.io/ICC490-1-P00/>
- [2] Cómo vincular un proyecto Java en IntelliJ IDEA con GitHub. <https://github.com/JesusTM/ICC490-1-P00/blob/main/2.Reportes/%5BRT-P00-01%5D%20GitHub%20-%20IntelliJIDEA.pdf>
- [3] Cómo usar Git y GitHub. <https://github.com/JesusTM/ICC490-1-P00/blob/main/2.Reportes/%5BRT-P00-02%5D%20Uso%20de%20GitHub.pdf>
- [4] Cómo crear un menú en Java (ciclo do-while). <https://tutospoo.jimdofree.com/tutoriales-java/estructuras-repetitivas/ciclo-do-while-1-men%C3%BA/>